

Spark DataFrames: Takeaways

by Dataquest Labs, Inc.

Syntax

- Creating a `SparkSession` :

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("MyApp").getOrCreate()
```

- Reading JSON data into a DataFrame:

```
df = spark.read.json("data.json")
```

- Exploring DataFrame structure:

```
df.printSchema()
df.show(10)
column_names = df.columns
column_types = df.dtypes
```

- Selecting columns:

```
df.select("age", "total").show()
```

- Creating new columns:

```
from pyspark.sql.functions import col
df_new = df.withColumn("ratio", col("females") / col("males"))
```

- Filtering rows:

```
filtered_df = df.filter(col("age") < 18)
```

- Aggregating data:

```
from pyspark.sql.functions import sum, avg, when
summary = df.groupBy("category").agg(sum("total").alias("total_pop"))
```

- Creating conditional columns:

```
df_categories = df.withColumn("group",
    when(col("age") < 18, "Minor")
    .when(col("age") < 65, "Working Age")
    .otherwise("Senior"))
```

- Chaining operations:

```
result = (df
    .withColumn("new_col", col("col1") / col("col2"))
    .filter(col("new_col") > 1.0)
    .select("age", "new_col")
    .orderBy("age")
)
```

- Converting to pandas for visualization:

```
pandas_df = spark_df.toPandas()
```

Concepts

- **DataFrames** are Spark's high-level abstraction for structured data with named columns and known data types, providing automatic optimization through the Catalyst engine.
- **Column objects** represent references to DataFrame columns and operations to perform on them. They're returned by `df.column_name` or `df["column_name"]` but can't be displayed directly.
- **Schema awareness** means DataFrames understand column names, data types, and constraints, enabling better performance, error detection, and code clarity compared to RDDs.
- **Immutability** ensures DataFrames cannot be modified after creation. Every transformation returns a new DataFrame, enabling thread safety, fault tolerance, and optimization opportunities.
- **Lazy evaluation** means transformations like `.select()`, `.filter()`, and `.withColumn()` build execution plans without immediate computation. Only actions like `.show()`, `.count()`, and `.collect()` trigger execution.
- **Method chaining** creates readable data processing pipelines by connecting transformations in sequence, with each step returning a DataFrame for the next operation.
- **PySpark-to-pandas conversion** enables visualization workflows where you process large datasets with PySpark's distributed capabilities, then convert summarized results to pandas for plotting with matplotlib.

Resources

- [PySpark DataFrame API Documentation](#)
- [DataFrame Programming Guide](#)
- [Understanding Spark Sessions](#)
- [Catalyst Optimizer Overview](#)
- [U.S. Census Dataset Reference](#)